

EpiSAM Demo: An Interactive System for Character Segmentation and Text-Line Extraction in Challenging Stone Inscriptions

Arnav Sharma, Pratyush Jena, Amal Joseph, and Ravi Kiran Sarvadevabhatla

Center for Visual Information Technology, IIIT Hyderabad, India
{arnav.sharma, pratyush.jena, amal.joseph}@research.iiit.ac.in
ravi.kiran@iiit.ac.in
<https://ihdia.iiit.ac.in/episam/>

Abstract. Stone inscriptions are invaluable sources of historical and linguistic knowledge, yet their automated analysis remains highly challenging due to surface irregularities, erosion, weathering and low visual contrast. Consequently, conventional document image analysis and handwriting recognition techniques often perform poorly on such artifacts. EpiSAM introduced a prompt-guided framework for character segmentation by jointly predicting a target character and its neighboring characters, leveraging local context to resolve ambiguous character boundaries and improve segmentation accuracy. In this demonstration, we extend EpiSAM with a lightweight text-line extraction method that groups segmented characters into text lines while introducing minimal additional computational overhead. The integrated system enables efficient character and line-level annotation, reducing the manual effort required by epigraphers while facilitating the large-scale digitization and analysis of stone inscriptions. **The demo video can be found at this [link](#).**

Keywords: Epigraphy · Character Segmentation · Text Line Detection · Interactive Demo · Segment Anything Model · EpiSAM

1 System Overview

The proposed system extends EpiSAM [1] by integrating automatic text-line formation and an interactive annotation interface into a unified end-to-end workflow for stone inscription analysis. Given a raw inscription image, the system first predicts character masks using EpiSAM, groups the segmented characters into text lines through a graph-based algorithm, and finally presents the results in an interactive interface for efficient human correction and annotation. Figure 1 illustrates the overall pipeline.

2 Technical Specifications

2.1 Binarization Module

Engraved characters often exhibit very low contrast against the surrounding stone, making reliable foreground extraction essential before segmentation. We therefore employ the Attention U-Net binarization model proposed in [2], which converts the input RGB inscription into a binary foreground mask that is subsequently used to generate prompts for EpiSAM.

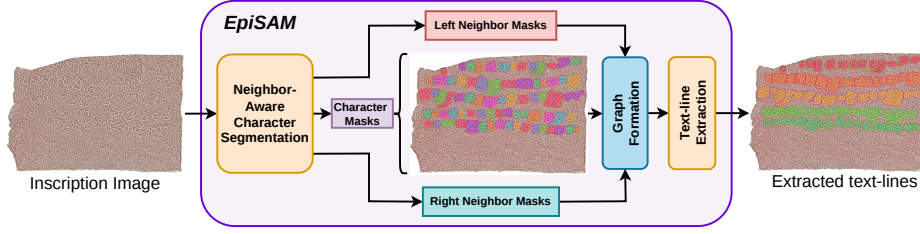


Fig. 1: Overview of the EpiSAM Character Segmentation and Text-line Extraction system

2.2 Character Segmentation Module

Following binarization, foreground connected components are extracted and converted into prompts for EpiSAM. Each prompt consists of the centroid and bounding box of the connected component. Components whose area is less than 10% of the mean connected-component area are discarded to suppress noise.

For every prompt, the Neighbor-Aware Decoder predicts three candidate target masks together with left- and right-neighbor masks. Predicted IoU scores are used to select the best target mask, while predictions with predicted IoU below 0.5 are discarded.

2.3 Text-Line Formation via Two-Hop Graph Construction

Having obtained a set of character masks from the segmentation stage, we group them into text lines using a graph-based approach. Each segmented character is treated as a node; edges are drawn between characters that may belong to the same line, and a threshold classifier then partitions the graph into line groups.

Step 1: 1-Hop Nearest-Neighbor Graph For each character node i , we compute the L2 distances from its centroid to all other character centroids and connect i to its k nearest neighbors ($k = 5$), forming the set of 1-hop edges.

Step 2: 2-Hop Extension Relying on 1-hop nearest-neighbor edges alone is fragile: a single missing or degraded character breaks the chain, splitting one line into multiple components. To avoid this, we add 2-hop edges: for each 1-hop edge (i, j) , we connect i to every 1-hop neighbor of j that is not already a 1-hop neighbor of i . The resulting graph contains both direct neighbors and neighbors-of-neighbors, giving each node broader connectivity that can span missing characters.

Step 3: Symmetric Similarity Score Not all edges connect characters on the same line; 2-hop edges especially may link characters across different lines. We therefore give each edge (i, j) a score that measures how well the two characters' predicted masks agree that they lie on the same line. EpiSAM [1] outputs three masks per character: the target mask T (the character itself), the left-neighbor prediction L , and the right-neighbor prediction R . We measure overlap between

two masks using Intersection-over-Union (IoU), written $\text{sim}(\cdot, \cdot)$ and take the edge score $s(i, j)$ as the maximum of seven overlap terms:

$$s(i, j) = \max(\text{sim}(T_i, T_j), \text{sim}(L_i, T_j), \text{sim}(R_i, T_j), \\ \text{sim}(T_i, L_j), \text{sim}(T_i, R_j), \\ \text{sim}(L_i, R_j), \text{sim}(R_i, L_j)). \quad (1)$$

The first five terms handle characters that are directly adjacent. $\text{sim}(T_i, T_j)$ is high when the two characters overlap directly. The other four compare one character’s target mask against the other’s neighbor prediction: the score is high when j ’s mask overlaps the region where i predicted its neighbor, or symmetrically when i ’s mask overlaps j ’s neighbor prediction.

The last two terms, $\text{sim}(L_i, R_j)$ and $\text{sim}(R_i, L_j)$, bridge gaps. When a character between i and j is missing, both nodes’ neighbor predictions lie in the same empty spot, producing a non-zero overlap score even though no character was present.

Step 4: Edge Classification with Common-Neighbor Veto

1. **Threshold rejection.** If $s(i, j) \leq \tau$, the edge is rejected immediately.
2. **Common-neighbor veto.** If the edge passes Rule 1, we examine every node c that is a graph neighbor of both i and j (available from the 2-hop construction). If c is strongly connected to one endpoint but not the other—that is, $s(i, c) \leq \tau$ while $s(j, c) > \tau$, or vice versa—the edge (i, j) is vetoed and labeled 0. The intuition is that c provides contradicting evidence: c is firmly connected to j , suggesting they share a line, but c is disconnected from i , suggesting i is on a different line. Accepting (i, j) would therefore incorrectly merge two distinct lines.
3. **Acceptance.** If neither rule triggers, the edge is labeled 1.

Finally, text lines are recovered as the connected components of the graph induced by the accepted (label 1) edges, found via breadth-first search over all nodes.

3 Interactive Demo Interface

3.1 Annotation Interface

The EpiSAM annotation platform is a browser-based, full-stack application designed to support the complete workflow from raw inscription image to a structured, transcribed dataset. The frontend is built with React 18 and TypeScript (Vite), with Konva/react-konva for hardware-accelerated canvas rendering, Zustand for state, and Axios over a REST API.

The backend uses FastAPI with SQLAlchemy over SQLite (PostgreSQL in production), containerised with Docker Compose behind Nginx.

The interface organizes the annotation workflow into three sequential modes, accessible through a tabbed toolbar. The main annotation interface is shown in Fig. 2.

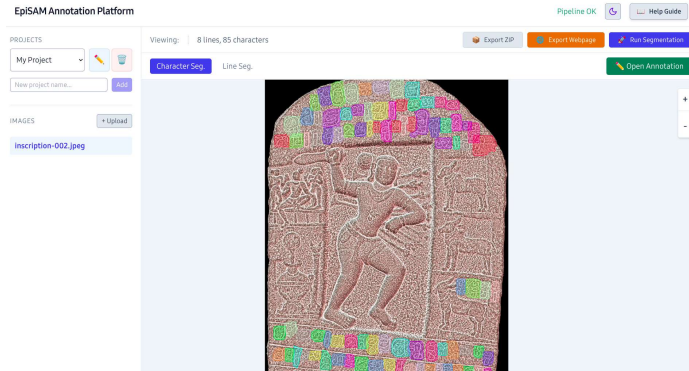


Fig. 2: The EpiSAM interactive annotation interface.

Character Mode. After the segmentation pipeline completes, predicted character masks are rendered as semi-transparent polygons on an interactive canvas that supports mouse-wheel zoom and middle-button pan. Annotators can select individual masks with a single click or build multi-selections with **Ctrl+Click**. Masks can be deleted with the **Delete** key. A polygon-draw tool allows annotators to manually trace regions that the model missed or only partially segmented. Where the model splits a single letter into two distinct masks—a common failure mode for letters with attached vowel signs—annotators can use the merge option to combine the selected masks into a single character mask. The character mode GUI is shown in Fig. 3.

Line Mode. In line mode, each character is colored according to its initially assigned line group. Rather than requiring annotators to click individual characters, a freehand *line curve* tool lets the user draw a continuous stroke through the text flow of a line; the system automatically detects all character masks intersected by the stroke, assigns them a shared line identifier, and determines their left-to-right reading order by horizontal centroid position. Line assignments can be corrected incrementally: redrawing a curve through an already-grouped line dynamically regroupes it. A single button re-indexes all lines by vertical position, normalizing the reading order from top to bottom. The GUI for the line annotation mode is shown in Fig. 4.

Assignment Mode. The assignment view transitions to a dual-panel layout. The left panel shows the full inscription canvas with the currently active character highlighted with a white border. The right panel displays a high-contrast crop of that character alongside input controls for transcription. Annotators type the corresponding Kannada grapheme with the help of a keyboard; pressing **Enter** commits the label and automatically advances to the next character, keeping the hands on the keyboard throughout. One-click buttons cover the common vowels as well as two special states: *NAL* (Not A Letter), for non-alphabetical caricatures, and *Uncertain*, for heavily degraded regions. The reading order within a line can be manually adjusted in cases where script conventions deviate from

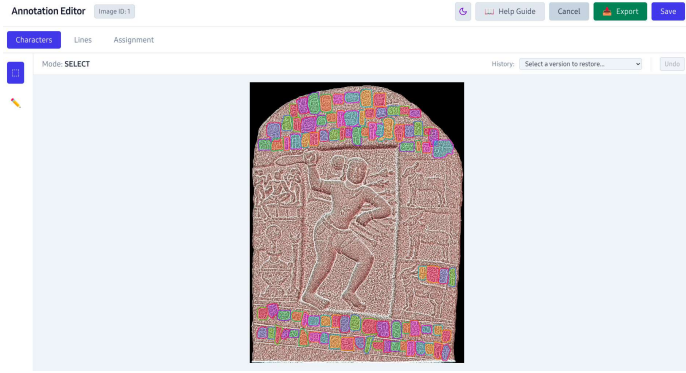


Fig. 3: Character Annotation Mode

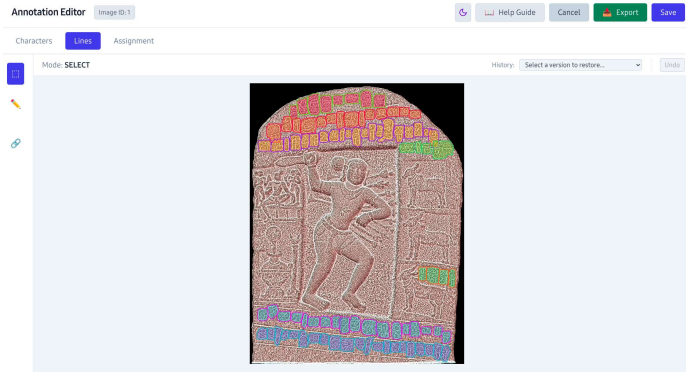


Fig. 4: Line Annotation Mode

strict left-to-right order. Refer to Fig. 5 for the GUI of the character assignment mode.

Export. Completed annotations can be exported as a structured ZIP archive containing a `metadata.json` record (bounding boxes, polygons, centroids, line groupings, and Kannada labels) together with the original image and individual character and line crops. Users can also generate a self-contained HTML webpage that renders the inscription image alongside a line-by-line grid of character crops and their transcriptions, suitable for sharing with collaborators or for peer review without requiring any additional software.

3.2 Epigraphic Research Workflows

In a typical epigraphic workflow, high-resolution images of stone inscriptions are captured in the field and subsequently examined by domain experts for manual transcription, a process that is both time-consuming and reliant on scarce specialist expertise. EpiSAM fits directly into this workflow: field images are

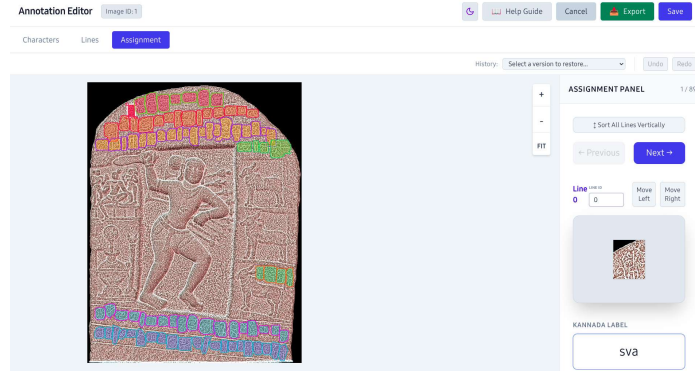


Fig. 5: Character Assignment Mode

uploaded to the platform, the pipeline automatically extracts character masks and line groupings, and a human expert then reviews and corrects the output using the interactive annotation interface. The corrected masks can be used for downstream tasks such as transcription. By automating the initial segmentation, EpiSAM significantly reduces the annotation bottleneck, allowing epigraphers to focus their effort on verification and transcription rather than on laborious boundary tracing.

3.3 Heritage Digitization at Scale

A large number of historical stone inscriptions remain undigitized because manual character annotation is prohibitively time consuming. By automatically producing character masks, line groupings, and editable annotations, our system substantially reduces the human effort required to create high-quality epigraphic datasets.

4 Conclusion

We present the EpiSAM annotation platform, an end-to-end interactive system for character and text-line extraction in ancient stone inscriptions. It extends the core EpiSAM method with a graph-based line-extraction method, and a full web-based annotation interface, enabling epigraphers to review, correct, and export structured annotation data with minimal manual effort.

Acknowledgments. We acknowledge The Mythic Society Bengaluru for providing the dataset images used in this research. We gratefully acknowledge the TCS Foundation’s support for Amal Joseph under the TCS Research Scholar Program.

References

1. Arnav Sharma, Pratyush Jena, Amal Joseph, and Ravi Kiran Sarvadevabhatla. Episam: Character segmentation in challenging stone inscriptions, 2026.
2. Pratyush Jena, Amal Joseph, Arnav Sharma, and Ravi Kiran Sarvadevabhatla. Unveiling text in challenging stone inscriptions: A character-context-aware patching strategy for binarization. *arXiv preprint arXiv:2601.03609*, 2026.